PostgreSQL con Docker Compose

Guia rapida para api-mi-ciudad y pgAdmin 4

Este proyecto ya incluye compose.postgres.yaml. Compose inicia dos servicios: la API y PostgreSQL 17. No hace falta instalar PostgreSQL por separado para que la API funcione; pgAdmin 4 se usa opcionalmente para explorar y administrar la base.

1. Iniciar la API y PostgreSQL

Abri Docker Desktop y una terminal PowerShell en la carpeta del proyecto. Si estas en la carpeta principal del repositorio, entra a api-mi-ciudad:

```
Set-Location ".\api-mi-ciudad"
docker compose -f compose.postgres.yaml up
```

La primera vez Docker puede tardar mientras descarga PostgreSQL y construye la imagen de la API. Deja esa terminal abierta: vas a ver los registros de los servicios. Cuando la API este lista, proba http://localhost:3000/salud. La documentacion esta en http://localhost:3000/docs.

Que significa el mapeo de puertos

PostgreSQL escucha en el puerto 5432 dentro de la red de Docker. Compose lo publica como 5433 en tu computadora. Por eso la API se conecta al servicio postgres por su nombre y puerto interno, mientras que pgAdmin 4 instalado en Windows se conecta a localhost:5433.

2. Conectar desde pgAdmin 4

Con Compose en ejecucion, abre pgAdmin 4. En el panel Browser, haz clic derecho en Servers y elige Register > Server. En General, escribe un nombre descriptivo, por ejemplo Mi Ciudad Docker.

En la pestana Connection completa estos datos:

- Host name/address: localhost
- Port: 5433
- Maintenance database: api
- Username: api
- Password: api
- Save password: opcional

Guarda con Save. Abre el servidor registrado y luego Databases > api. Desde ahi puedes explorar schemas y tablas, y abrir Query Tool para ejecutar SQL. Por ejemplo:

```
SELECT version();
```

La contrasena api es para desarrollo local, tal como esta configurado el Compose del proyecto. No reutilices estas credenciales en una base publicada.

Si pgAdmin tambien corre en Docker

Si pgAdmin 4 esta dentro de otro contenedor, localhost apunta a ese contenedor, no a Windows. En Docker Desktop para Windows, prueba host.docker.internal como host y 5433 como puerto. Si ambos contenedores comparten una red de Docker, tambien se puede conectar usando el nombre del servicio postgres y el puerto 5432.

3. Ejecutar en segundo plano y detener

Para dejar los servicios en segundo plano:

```
docker compose -f compose.postgres.yaml up -d
docker compose -f compose.postgres.yaml logs -f
```

El segundo comando muestra los registros; sal con Ctrl+C sin detener los contenedores. Para apagarlos, ejecuta:

```
docker compose -f compose.postgres.yaml down
```

Si los iniciaste sin -d, Ctrl+C detiene los servicios. En ambos casos, down detiene y elimina los contenedores, pero conserva el volumen de datos de PostgreSQL.

4. Empezar con una base vacia

```
docker compose -f compose.postgres.yaml down -v
```

El modificador -v elimina tambien el volumen de PostgreSQL y borra los datos guardados. Usalo solo si realmente quieres reiniciar la base desde cero.

Problemas frecuentes

- No conecta / Connection refused: verifica que Docker Desktop este abierto y que Compose siga ejecutandose.
- Authentication failed: revisa base api, usuario api y contrasena api.
- El puerto 5433 ya esta ocupado: en PowerShell ejecuta $env:PUERTO_DB="5434" y vuelve a iniciar Compose. En pgAdmin usa el puerto 5434.
- La API tarda en iniciar: espera a que PostgreSQL este saludable y revisa los registros con docker compose -f compose.postgres.yaml logs -f.

Resumen de conexion desde pgAdmin 4: localhost | puerto 5433 | base api | usuario api | contrasena api